

EVENT AND VENUE BOOKING PLATFORM

Backend API Documentation

Developed Using:

Spring Boot

Java

MySQL

JPA

Hibernate

Maven

Prepared By: Sreei Easwar S

1. Project Overview

The Event & Venue Booking Platform backend provides REST APIs for managing users, categories, venues, availability, carts, bookings, and checkout. The backend follows a layered architecture using Spring Boot and Spring Data JPA. APIs are tested using Postman.

2. Technology Stack

Java 21: Programming Language

Spring Boot: Backend Framework

Spring Data JPA: ORM

Hibernate: Persistence

MySQL: Database

Maven: Build Tool

Postman: API Testing

Git Version: Control

Event & Venue Booking Platform - Complete Flow

Think of it like BookMyShow, RedBus or a wedding hall booking website.

Users

- Customer
- Venue Owner
- Admin

Step 1: Admin creates Categories

Categories such as Birthday, Wedding, Corporate, Photography, Conference, Party Hall. Category supports CRUD operations.

Step 2: Venue Owner adds Venues

A venue contains: Name, Category, Price, Capacity, Location, Description, Services, Images. One Category can have many Venues.

Step 3: Availability

Each venue has date-wise available slots. Every successful booking decreases the slot count. If slots reach zero, further bookings are rejected to prevent overbooking.

Step 4: Search

Customers search by Category, Location, Date, Price Range, and Capacity.

Step 5: Cart

Customers add Venue + Date + Slot to the cart. Nothing is booked yet.

Step 6: Checkout

Validate availability, decrease slot count, create booking. If no slots are available, return an error.

Step 7: Booking

Booking contains Customer, Venue, Date, Slot, and Status (Pending, Confirmed, Cancelled).

Step 8: Views

Customer: My Bookings.

Venue Owner: Bookings for owned venues.

Admin: All bookings and status management.

Overall Flow

Admin creates Categories -> Venue Owner creates Venue -> Adds Availability -> Customer searches -> Adds to Cart -> Checkout -> Availability checked -> Booking created -> Customer/Owner/Admin view bookings.

Main Entities

- User
- Category
- Venue
- Availability
- Cart
- Booking

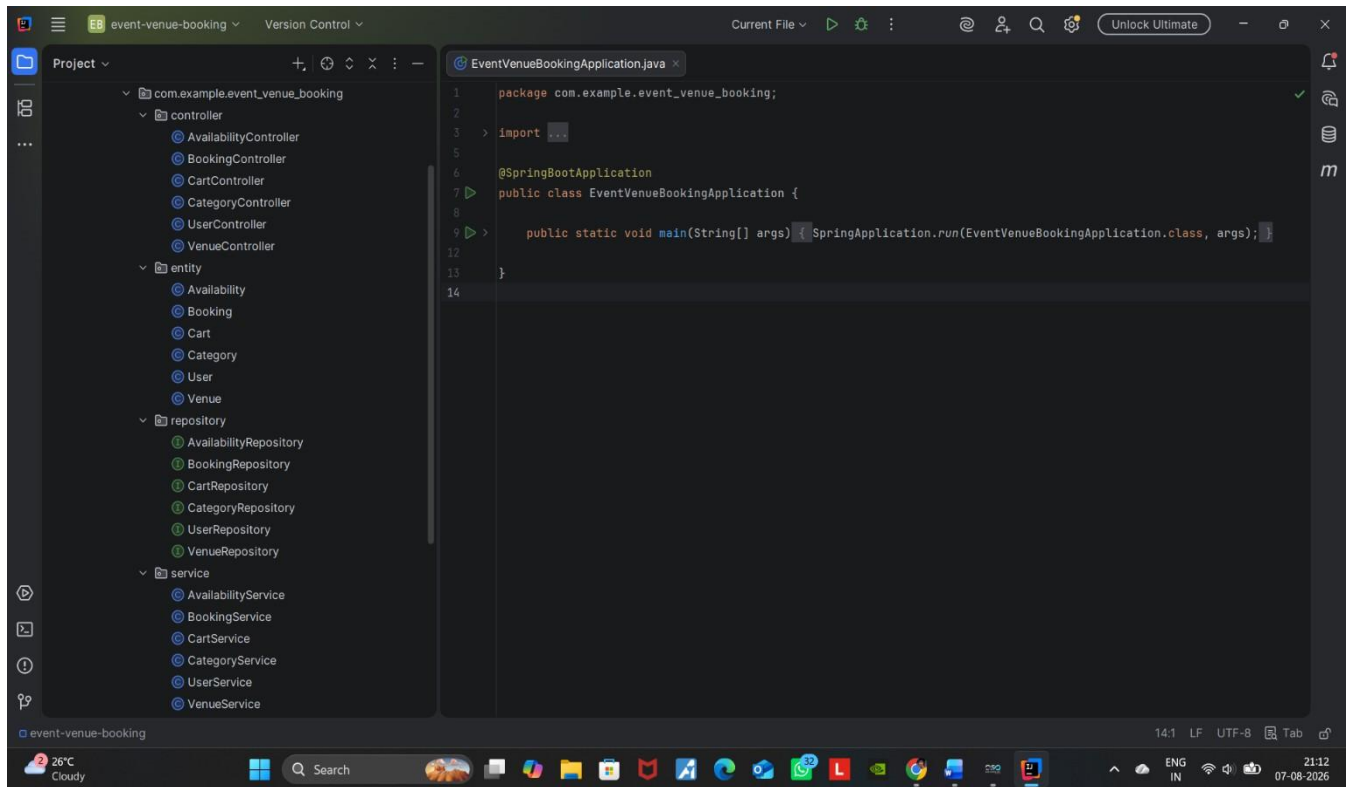
Relationships

User -> Category -> Venue -> Availability, Cart, Booking.

Spring Boot Structure

For each entity: Entity, Repository, Service, Controller.

3. Project Structure



4. API Documentation

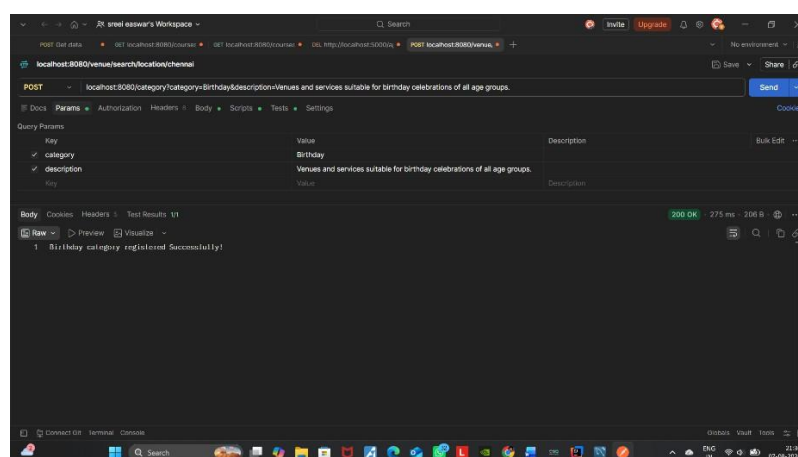
1. Category Entity

The Admin can Add/Delete/Update Categories in the Booking Platform.

- createCategory()—Post

End Point: localhost:8080/category

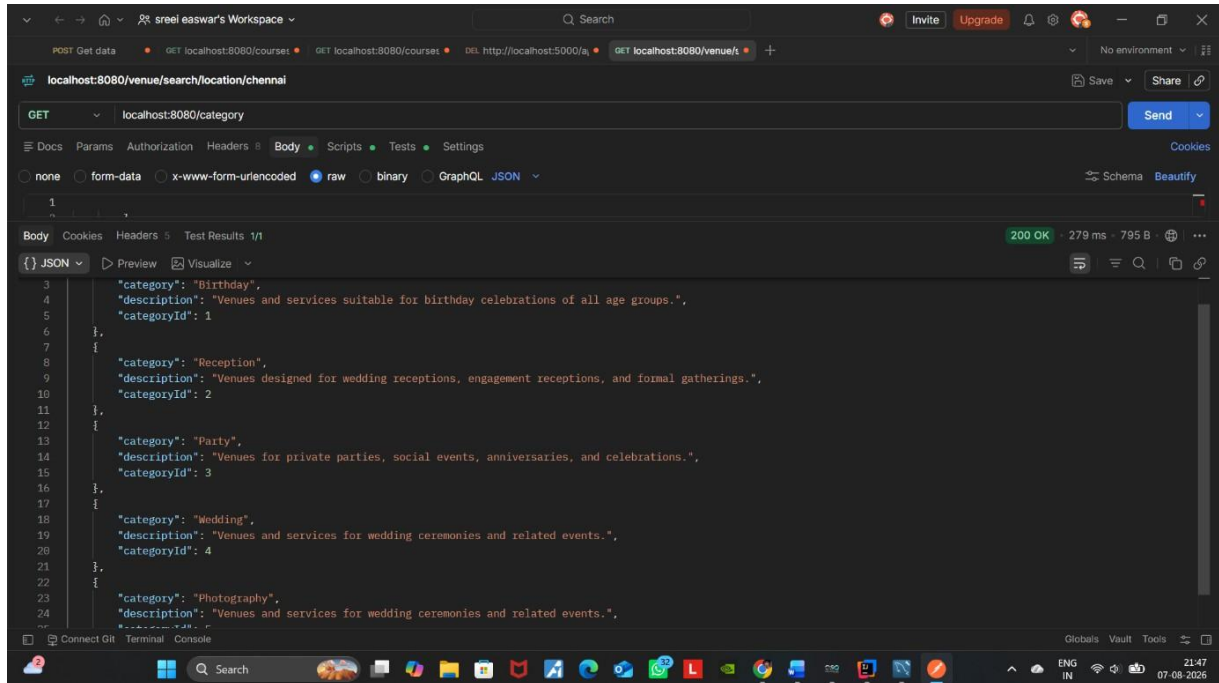
Description: To create a category



- **getCategories()—Get**

End Point: localhost:8080/category

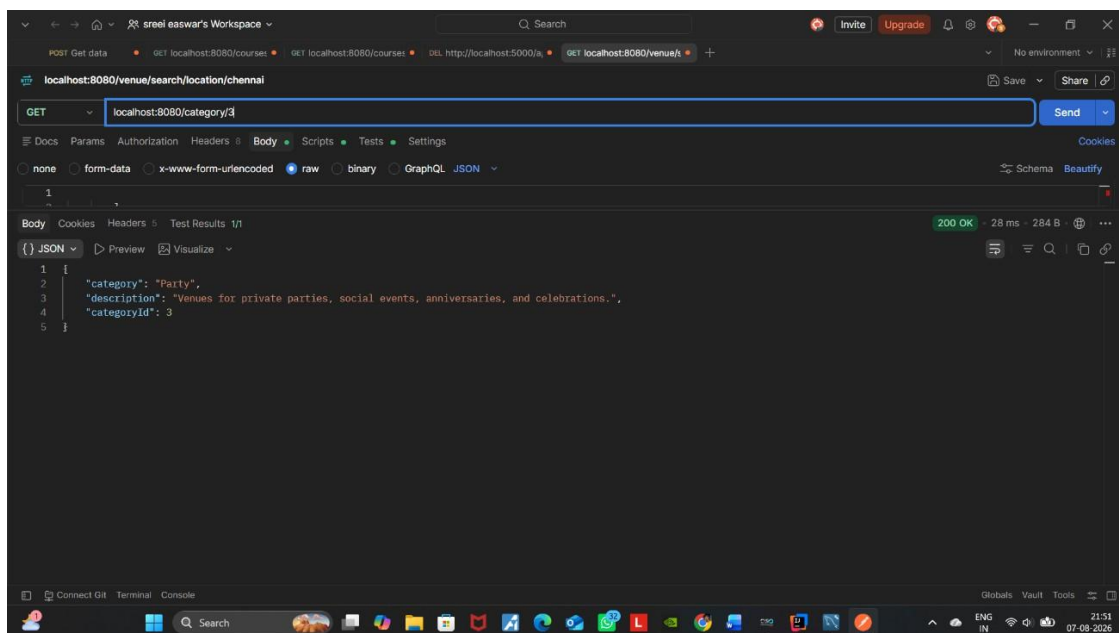
Description: To List all categories



- **getCategoryById()—Get**

End Point: localhost:8080/category/{categoryId}

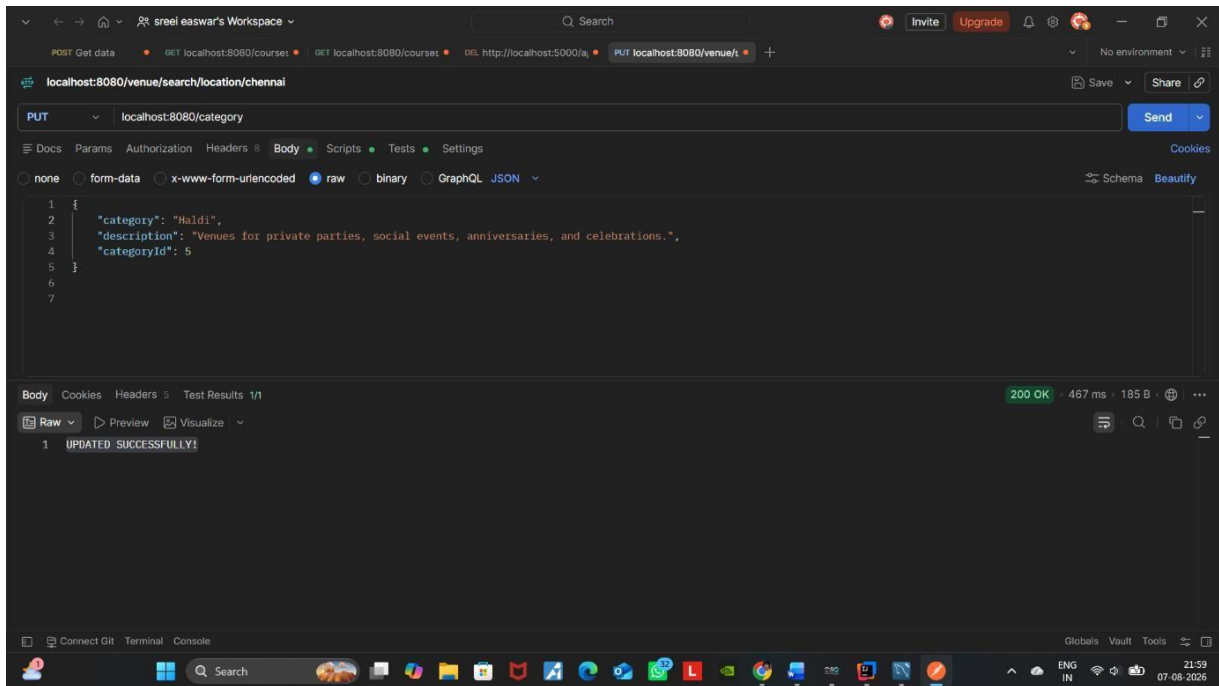
Description: To get a category by unique categoryId



- **updateCategory()—Put**

End Point: localhost:8080/category

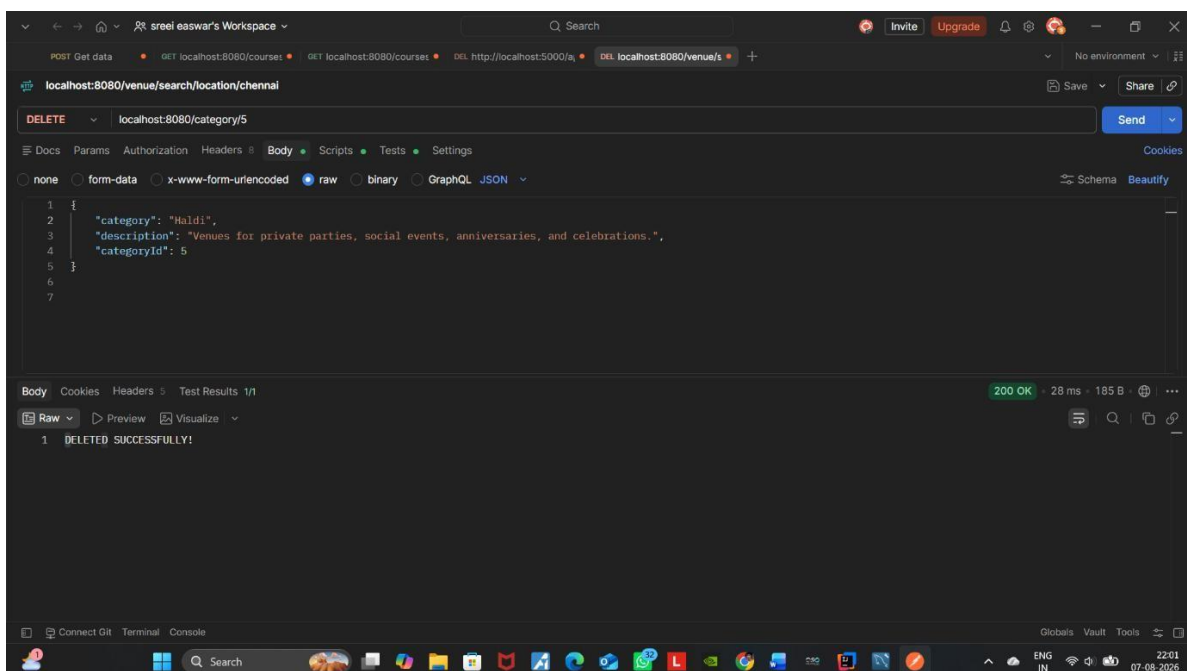
Description: To update a category



- **deleteCategoryById()—Delete**

End Point: localhost:8080/category/{categoryId}

Description: To delete a particular category



2. User Entity

The User can Create their own UserId in the venue booking platform. The user can be ADMIN or VENUEOWNER or CUSTOMER.

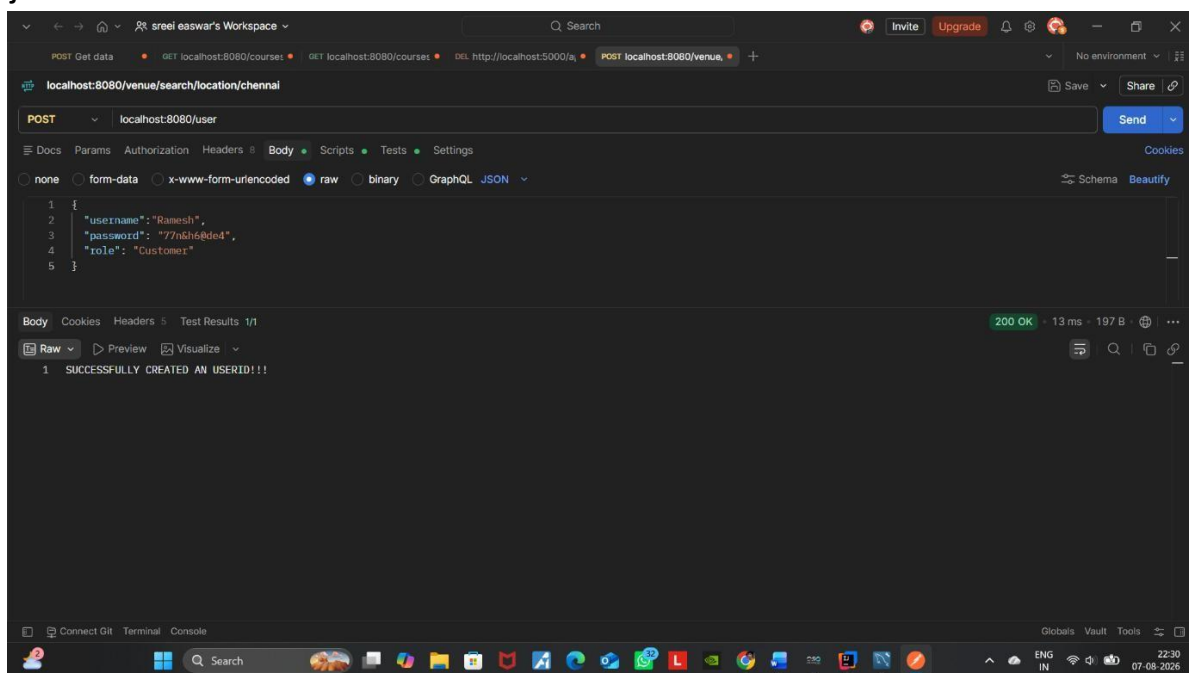
- **createUser()—Post**

End Point: localhost:8080/user

Description: To create a UserId

Request Body:

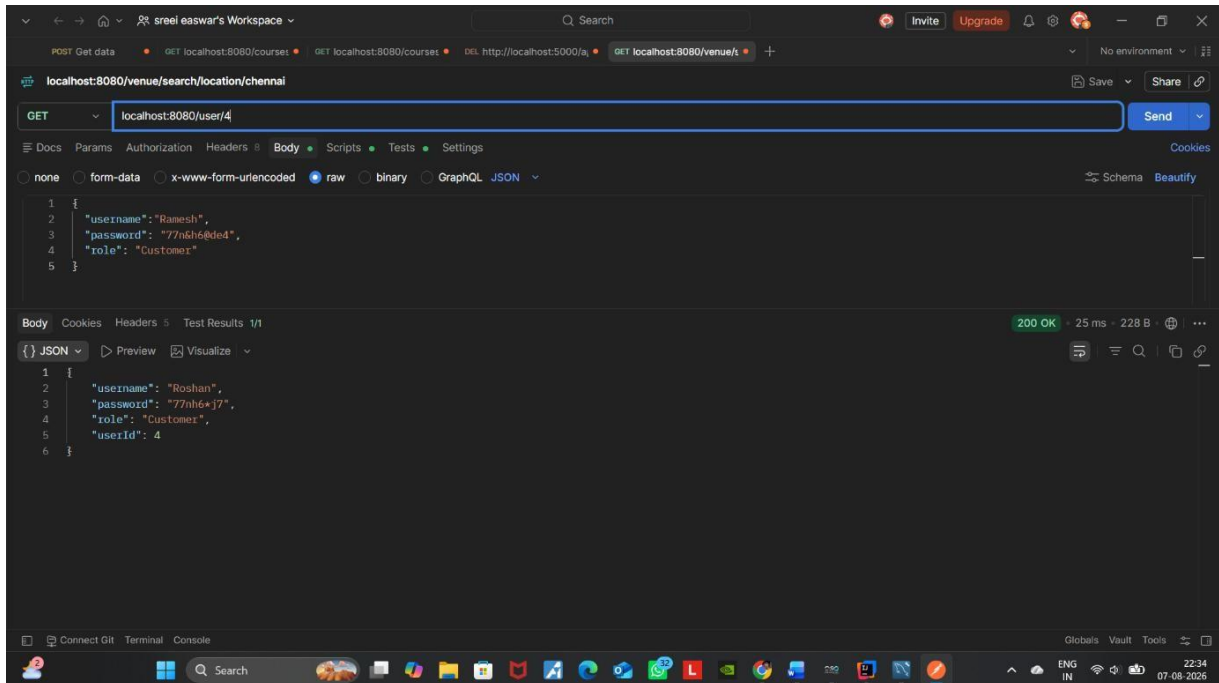
```
{
  "username": "Ramesh",
  "password": "77n&h6@de4",
  "role": "Customer"
}
```



- **getUserById()—Get**

End Point: localhost:8080/user/{user_Id}

Description: To create a UserId



3. Venue Entity

The VenueOwner can create, update and delete his own venue with respect to the availability.

- **createVenue()—Post**

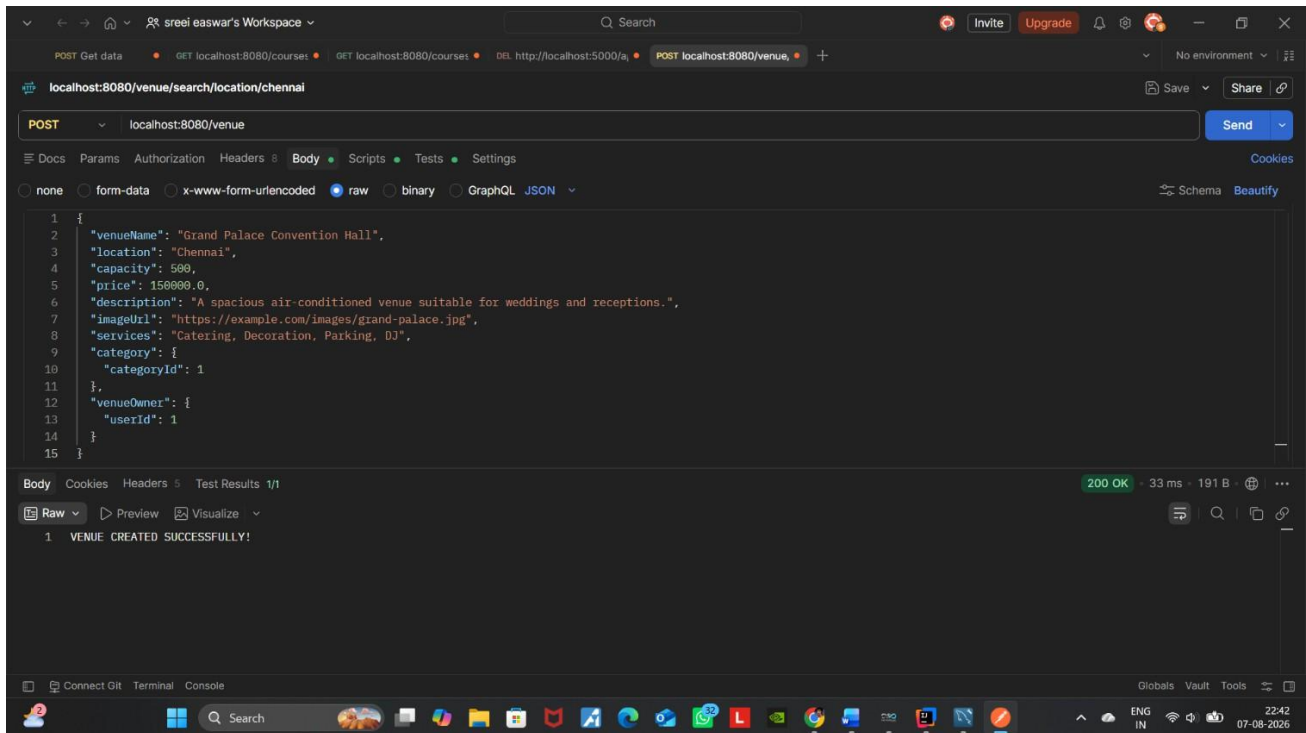
End Point: localhost:8080/venue

Description: To create a Venue

Request Body:

```
{
  "venueName": "Grand Palace Convention Hall",
  "location": "Chennai",
  "capacity": 500,
  "price": 150000.0,
  "description": "A spacious air-conditioned venue suitable for
weddings and receptions."}
```

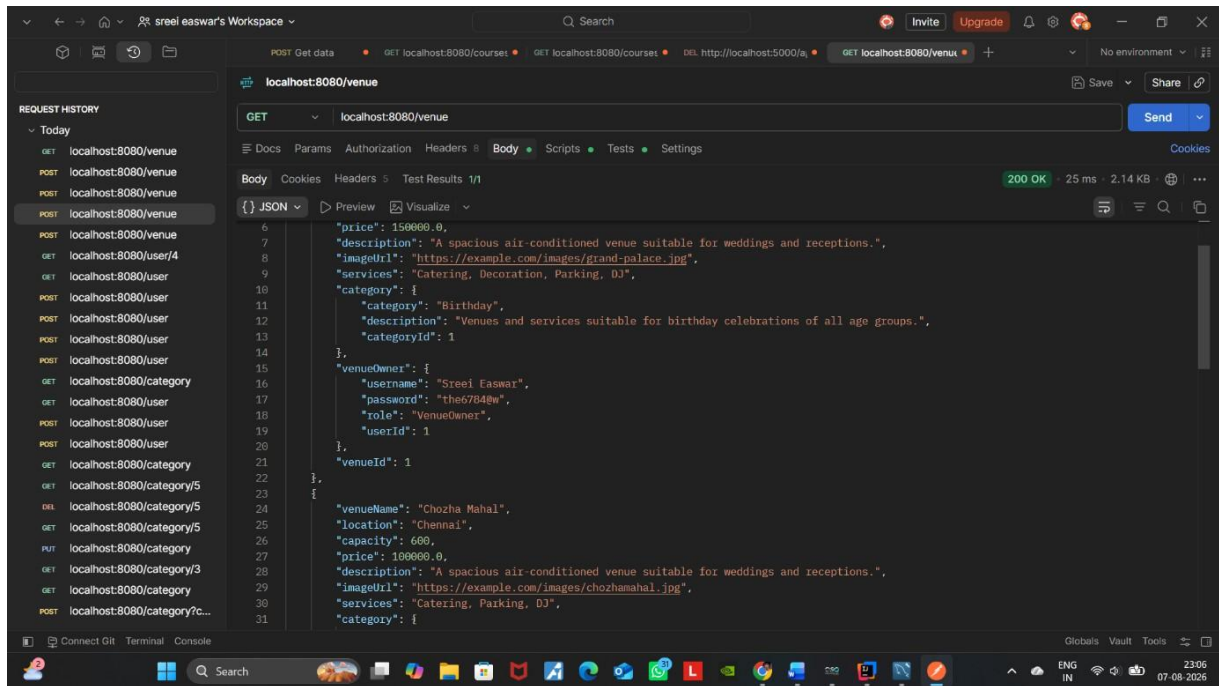
```
"imageUrl": "https://example.com/images/grand-palace.jpg",
"services": "Catering, Decoration, Parking, DJ",
"category": {
  "categoryId": 1
},
"venueOwner": {
  "userId": 1
}
}
```



- **getVenue()—get**

End Point: localhost:8080/venue

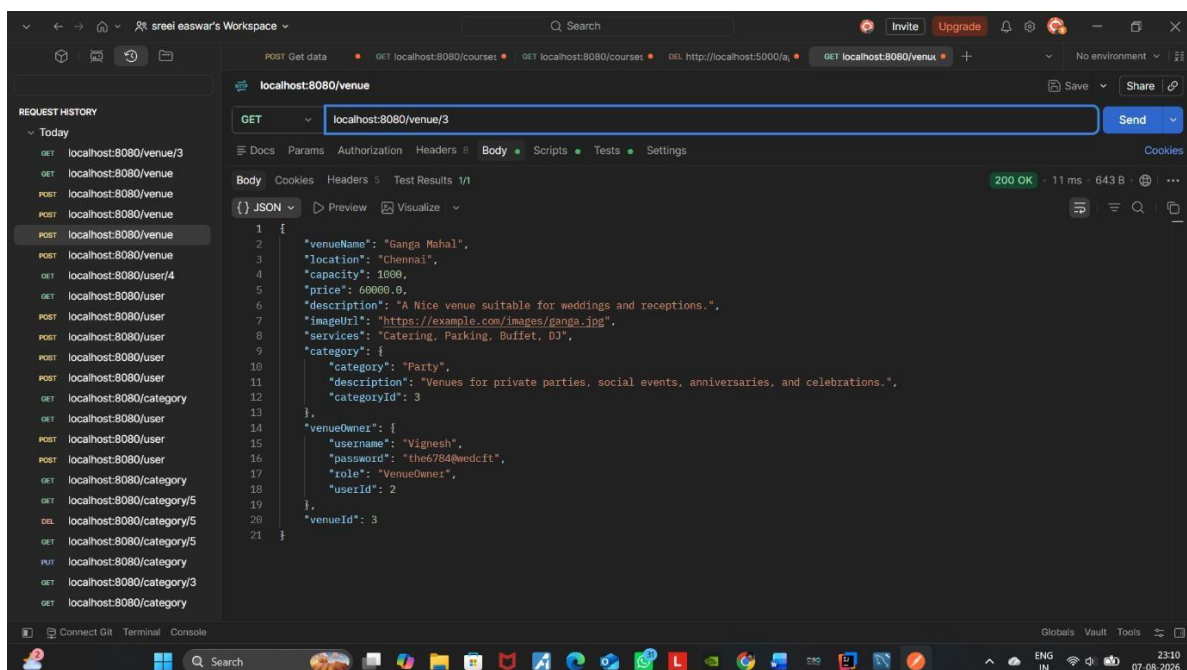
Description: To get all the Venues



- **getVenueById()—get**

End Point: localhost:8080/venue/{venueId}

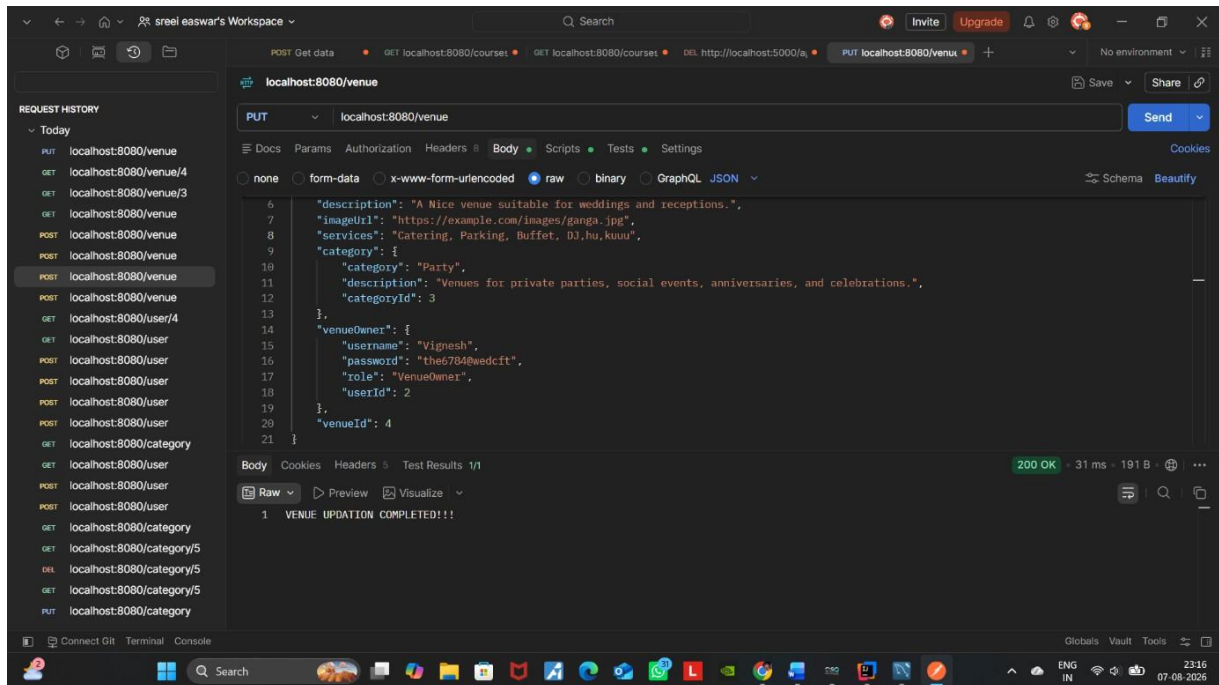
Description: To get a Venue by a unique VenueId



- **updateVenue()—Put**

End Point: localhost:8080/venue

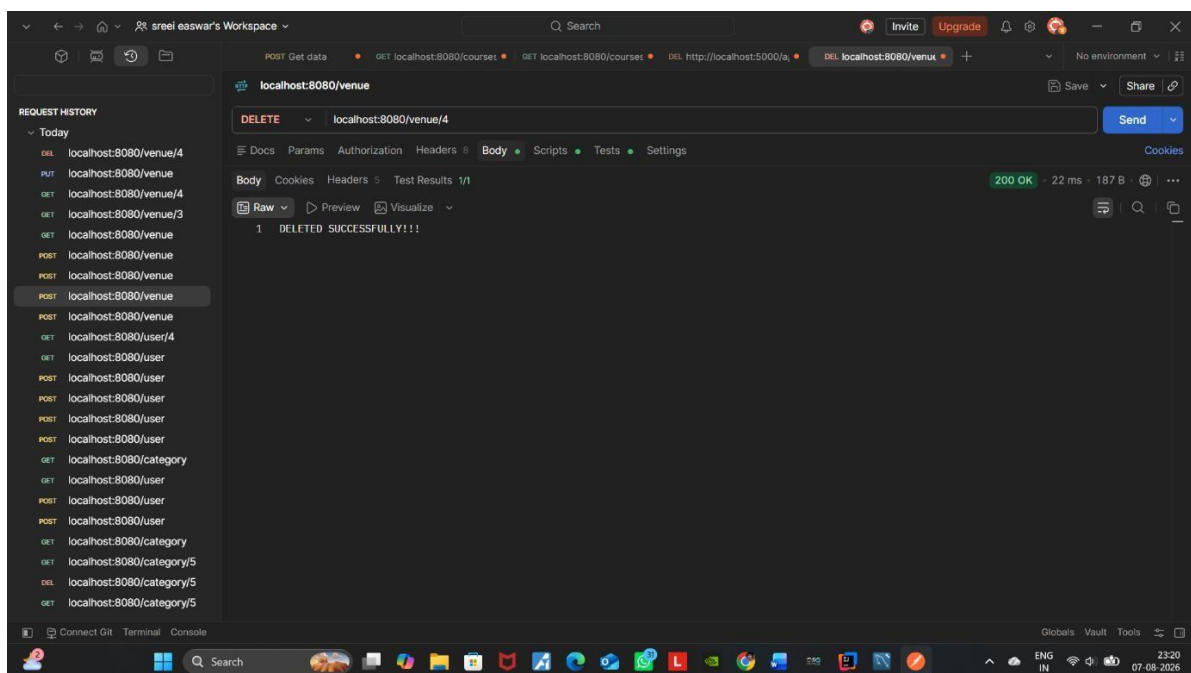
Description: To Update a Venue



- **deleteVenue()—Delete**

End Point: localhost:8080/venue/{venueId}

Description: To Delete a particular Venue by unique Id



4. Availability Entity

The Venue Owner Can update and create the availabilities for their respective venues.

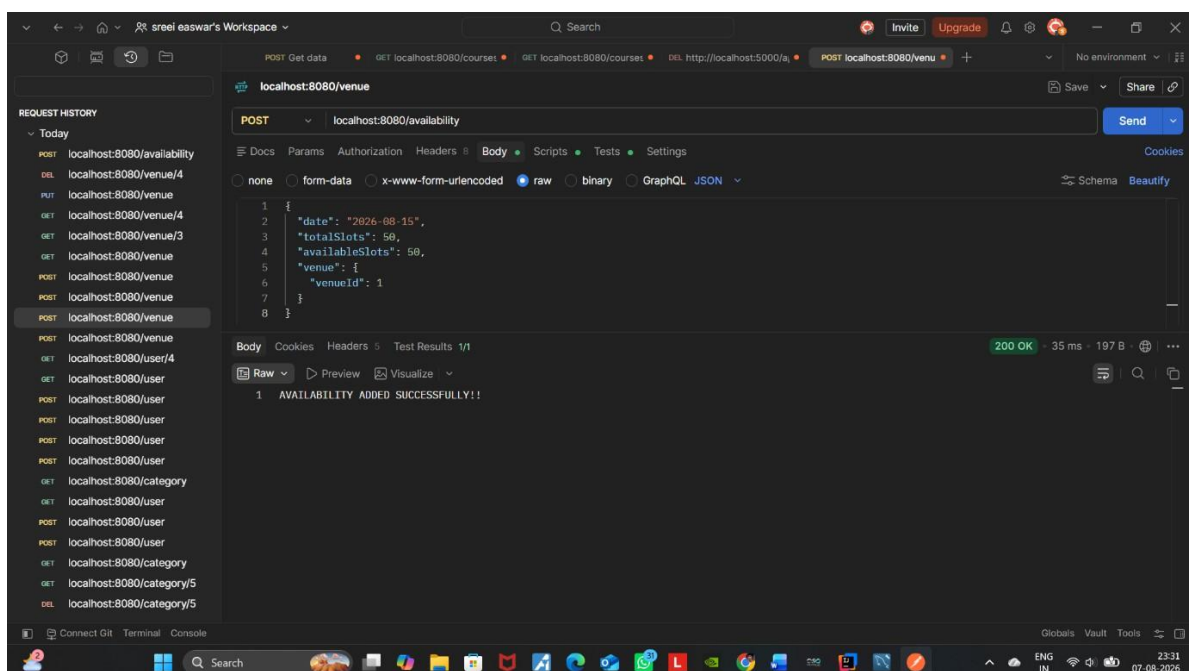
- **addAvailability()—Post**

End Point: localhost:8080/availability

Description: To create the availability for a particular venue.

Request Body:

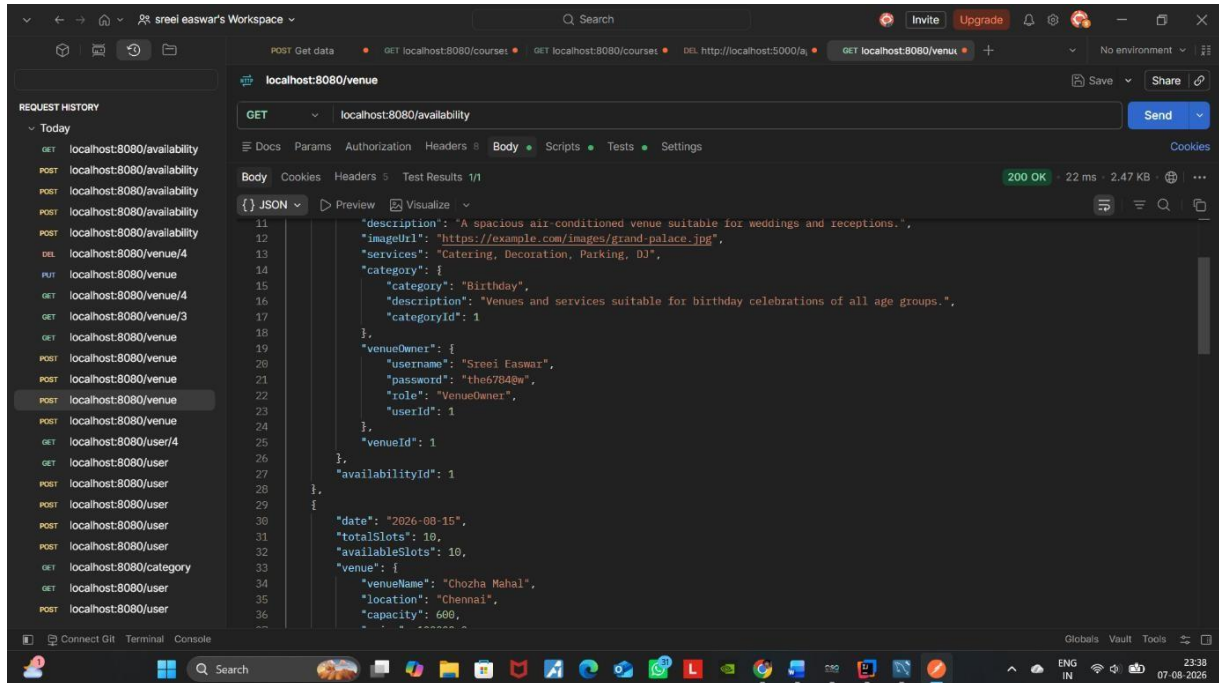
```
{  
  
  "date": "2026-08-15",  
  
  "totalSlots": 50,  
  
  "availableSlots": 50,  
  
  "venue": {  
  
    "venueId": 1  
  
  }  
  
}
```



- **getAvailabilities()—Get**

End Point: localhost:8080/availability

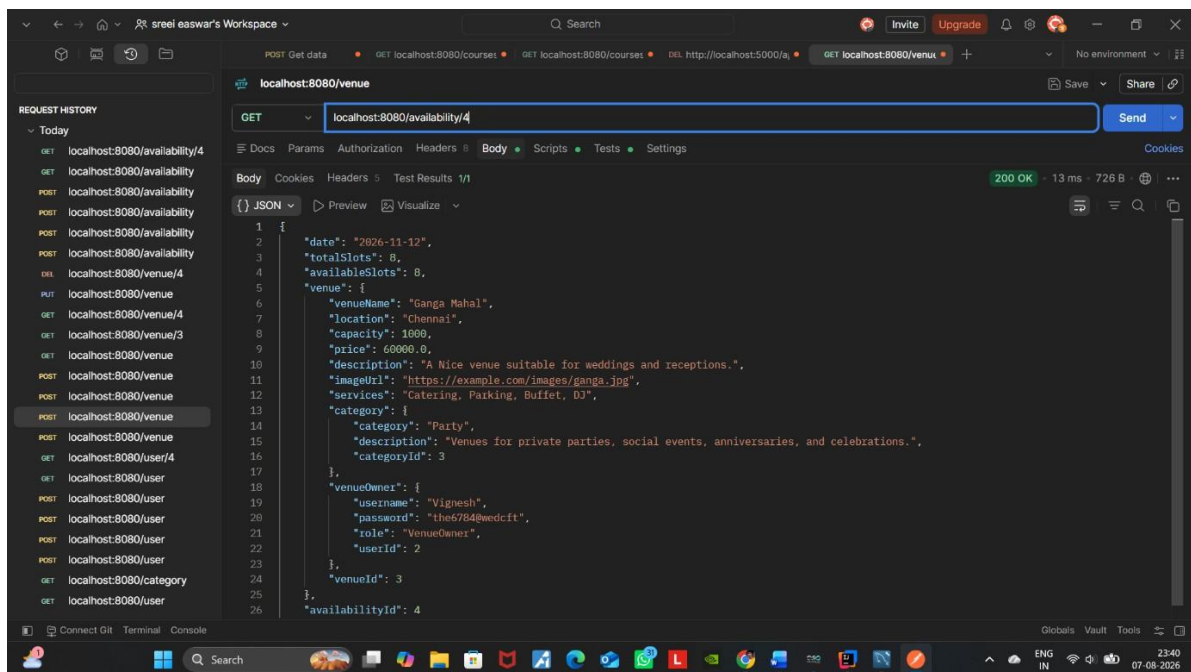
Description: To get all the availabilities(for admin).



- **getAvailabilityById()—Get**

End Point: localhost:8080/availability/{availabilityId}

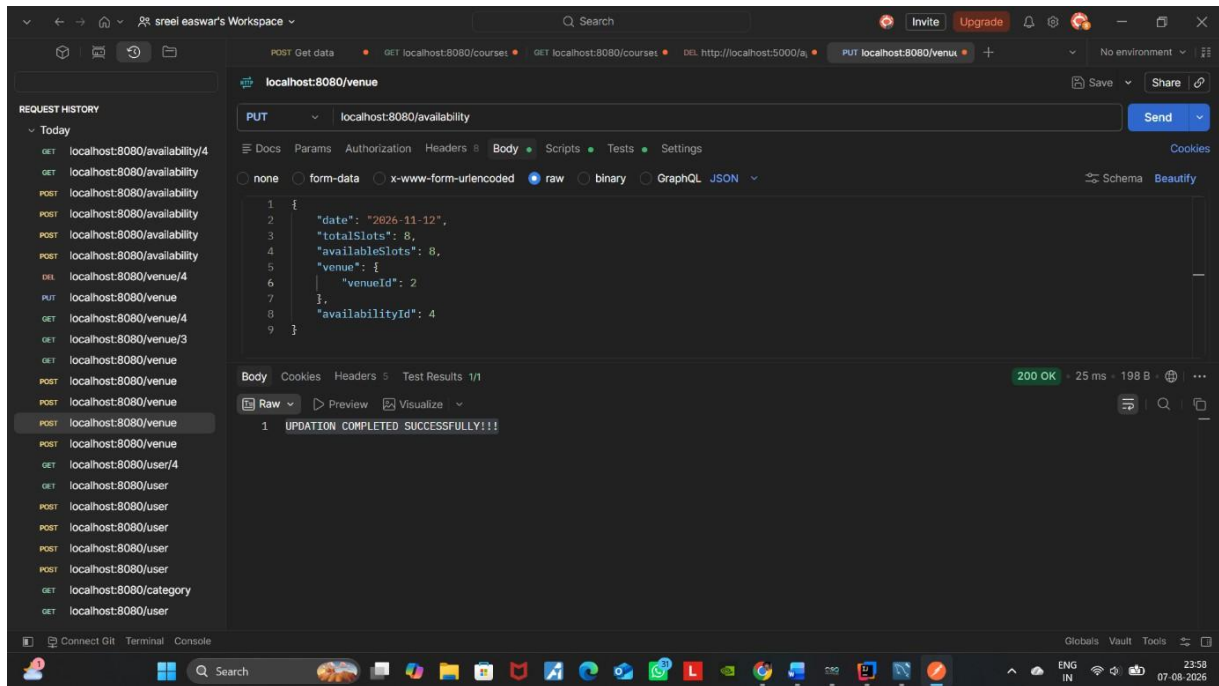
Description: To get a particular availability



- **updateAvailability()—Put**

End Point: localhost:8080/availability

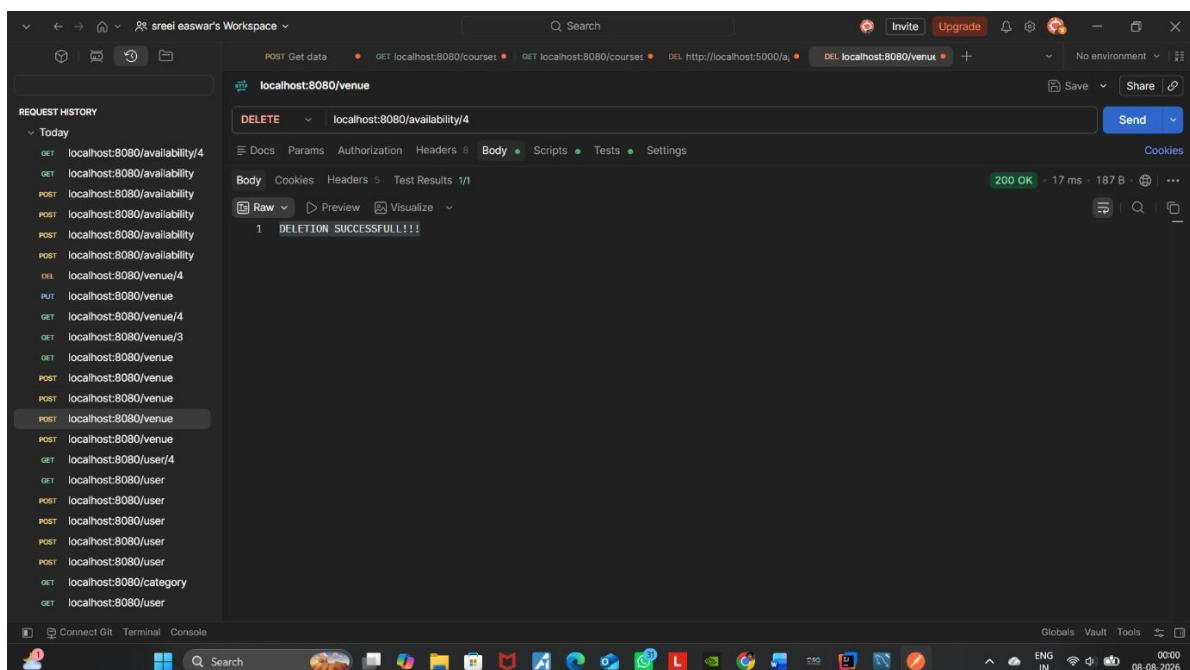
Description: To update an availability by the venue owner.



- **deleteAvailability()—Delete**

End Point: localhost:8080/availability/{availabilityId}

Description: To delete a particular availability.



5. Cart Entity

The Customer can add their venues to the cart and they can update it and book them.

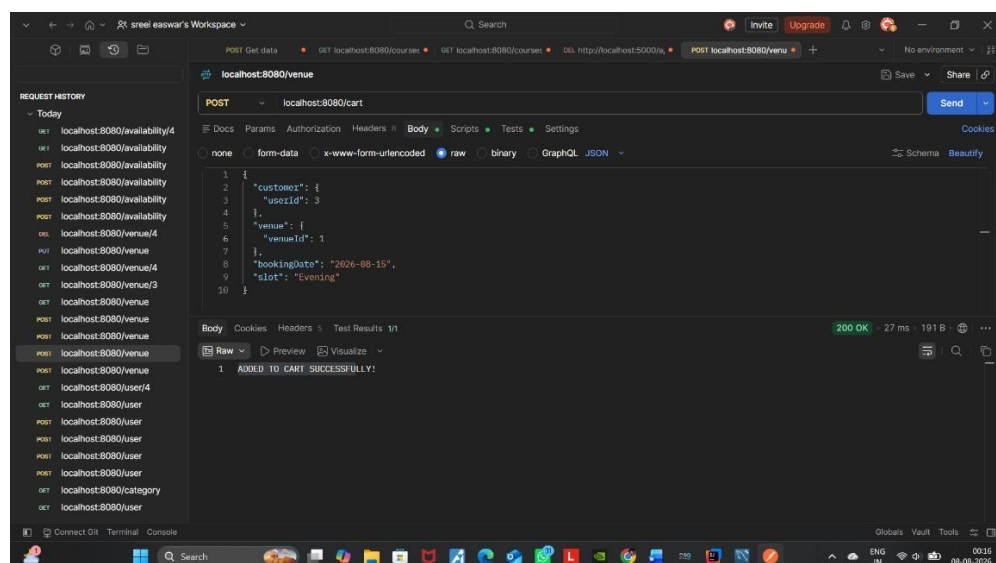
- **addToCart()**—Post

End Point: localhost:8080/cart

Description: To add the venue to the cart by the Customer.

Request Body:

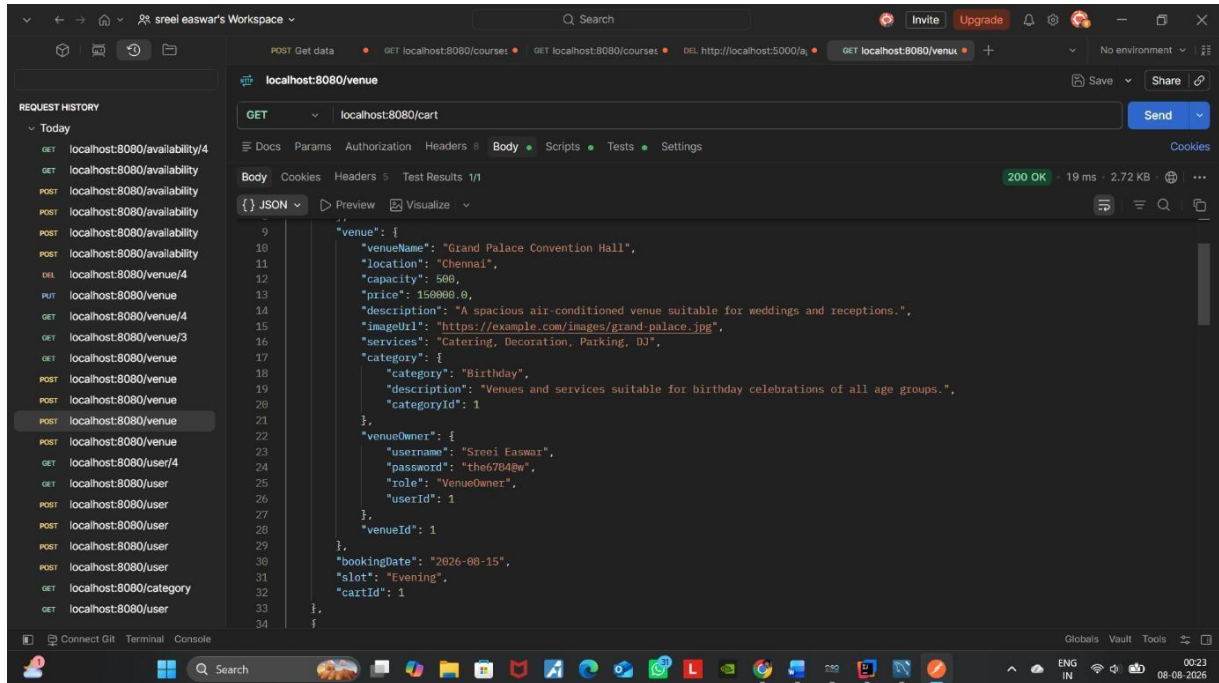
```
{  
  "customer": {  
    "userId": 3  
  },  
  "venue": {  
    "venueId": 1  
  },  
  "bookingDate": "2026-08-15",  
  "slot": "Evening"  
}
```



- **getAllCart()—Get**

End Point: localhost:8080/cart

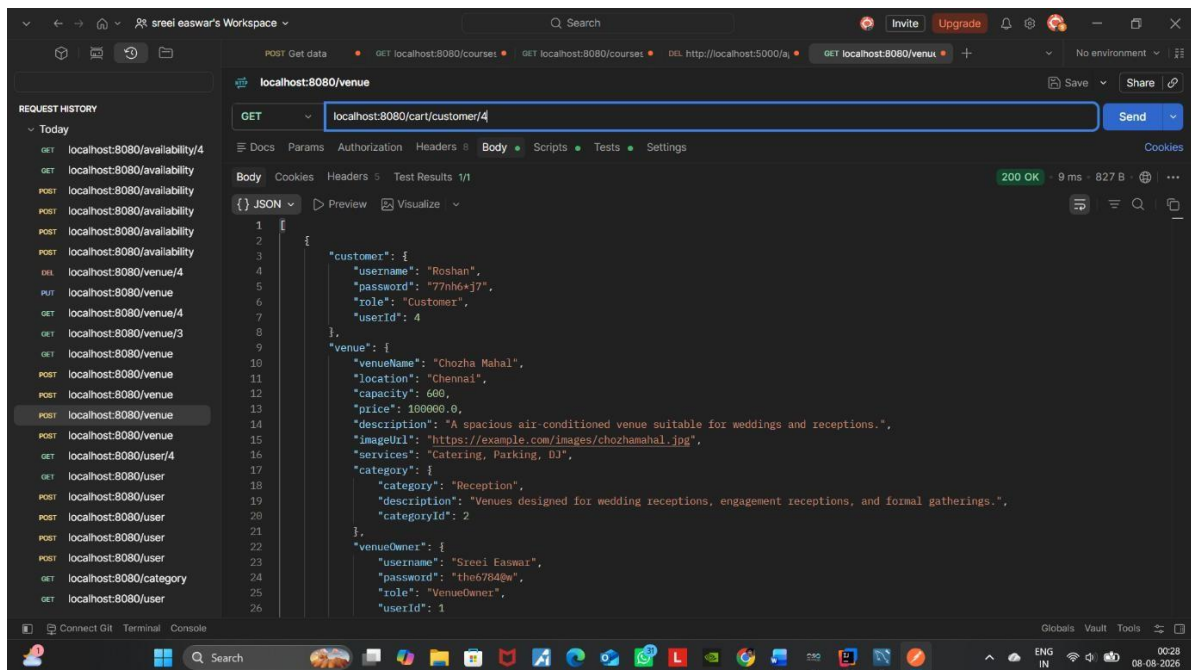
Description: To get all the carts.



- **getCartByCustomerId()—Get**

End Point: localhost:8080/cart/customer/{customerId}

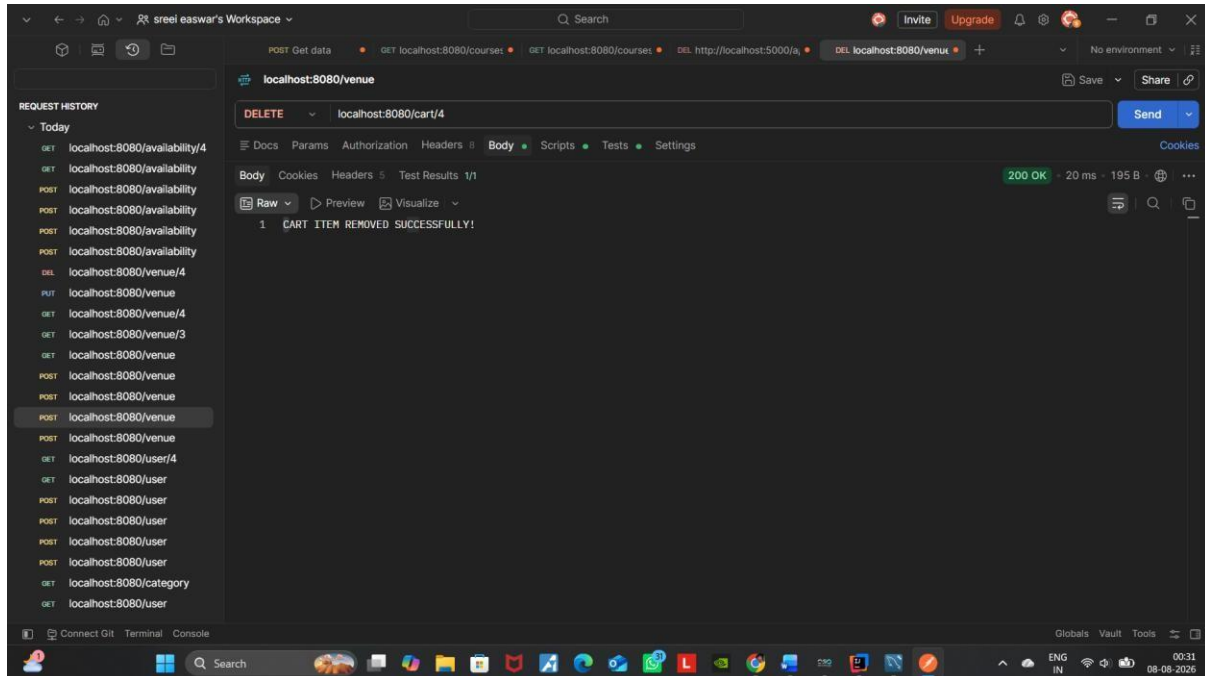
Description: To get the cart by customerId



- **deleteCart()—Delete**

End Point: localhost:8080/cart/{cartId}

Description: To delete the cart



6. Booking Entity

The Booking API does not require a JSON request body for checkout. Instead, the **Customer ID is passed as a path variable in the request URL**.

When the checkout API is called, the system uses the given userId to identify the customer. It then retrieves all items added by that customer from the cart and automatically uses the cart details to create the booking.

The system automatically:

- Retrieves the customer's cart items.
- Retrieves the availability for the selected venue and date.
- Checks whether slots are available.

- Decreases the available slot count by 1.
- Creates the booking using the cart details.
- Sets the booking status as CONFIRMED.
- Records the booking time automatically.
- Removes the cart items after successful checkout.

Customer ID



Checkout API



Find Customer



Get Customer's Cart



Get Venue & Availability



Check Available Slots



Decrease Available Slots



Create Booking



Status = CONFIRMED



Delete Cart Items

Example:

POST /bookings/checkout/1

1 → Customer ID

No JSON request body is required.

The booking details such as venue, booking date, slot, and amount are automatically obtained from the customer's cart. The totalAmount, status, and bookingTime are also automatically assigned by the backend.

Other Booking APIs:**1. Customer – View My Bookings**

The customer ID is passed in the URL. The system retrieves and displays all bookings belonging to that customer.

localhost:8080/customer/{userId}

2. Venue Owner – View Venue Bookings

The venue owner's ID is passed in the URL. The system retrieves all bookings associated with venues owned by that owner.

localhost:8080/owner/{ownerId}

3. Admin – View All Bookings

No request body or ID is required. The admin can retrieve all bookings.

localhost:8080/admin

POST Get data • GET localhost:8080/course • GET localhost:8080/course • DEL http://localhost:5000/a • GET localhost:8080/venue • No environment

localhost:8080/venue

GET localhost:8080/admin Send

Docs Params Authorization Headers Body Scripts Tests Settings Cookies

Body Cookies Headers Test Results 1/1

JSON Preview Visualize

```
200 OK • 14 ms • 1.62 KB
```

```
{  "userId": 3,  "venue": {    "venueName": "Ganga Mahal",    "location": "Chennai",    "capacity": 1000,    "price": 60000.0,    "description": "A Nice venue suitable for weddings and receptions.",    "imageUrl": "https://example.com/images/ganga.jpg",    "services": "Catering, Parking, Buffet, DJ",    "category": {      "category": "Party",      "description": "Venues for private parties, social events, anniversaries, and celebrations.",      "categoryId": 3    },    "venueOwner": {      "username": "Vignesh",      "password": "the6784@wddcft",      "role": "VenueOwner",      "userId": 2    }  },  "venueId": 3,  "bookingDate": "2026-11-12",  "slot": "Morning",  "totalAmount": 60000.0}
```

Connect Git Terminal Console

00:55 08-08-2026

POST Get data • GET localhost:8080/course • GET localhost:8080/course • DEL http://localhost:5000/a • GET localhost:8080/venue • No environment

localhost:8080/venue

GET localhost:8080/owner/1 Send

Docs Params Authorization Headers Body Scripts Tests Settings Cookies

Body Cookies Headers Test Results 1/1

JSON Preview Visualize

```
200 OK • 31 ms • 934 B
```

```
{  "customer": {    "username": "Salman",    "password": "thywedcft",    "role": "Customer",    "userId": 3  },  "venue": {    "venueName": "Grand Palace Convention Hall",    "location": "Chennai",    "capacity": 500,    "price": 150000.0,    "description": "A spacious air-conditioned venue suitable for weddings and receptions.",    "imageUrl": "https://example.com/images/grand-palace.jpg",    "services": "Catering, Decoration, Parking, DJ",    "category": {      "category": "Birthday",      "description": "Venues and services suitable for birthday celebrations of all age groups.",      "categoryId": 1    },    "venueOwner": {      "username": "Sree Easwar",      "password": "the6784@w",      "role": "VenueOwner",      "userId": 1    }  }
```

Connect Git Terminal Console

00:53 08-08-2026

5.Database(MySQL)

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHMAS

Filter objects

course_reg_sym

Tables

Views

Stored Procedures

Functions

event_booking

Tables

Views

Stored Procedures

Functions

jpdemo

sreeaswar

sys

SQL File 3*

SQL File 3* - %

Limit to 1000 rows

```
1 create database event_bookings;
2 use event_bookings;
3 show tables;
4 describe category;
5 select * from category;
6 describe venue;
7 select * from venue;
8 describe availability;
9 select * from availability;
10 describe user;
```

Result Grid

Filter Rows

Edit

Export/Import

Wrap Cell Contents

category_id	category	description
1	Birthday	Venues and services suitable for birthday celebr...
2	Reception	Venues designed for wedding receptions, enga...
3	Party	Venues for private parties, social events, anniv...
4	Wedding	Venues and services for wedding ceremonies an...

category 1 x

Apply Revert Context Help Snippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	20:25:43	use event_booking	0 row(s) affected	0.000 sec
2	20:25:51	select * from category LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec

Object Info Session

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHMAS

Filter objects

course_reg_sym

Tables

Views

Stored Procedures

Functions

event_booking

Tables

Views

Stored Procedures

Functions

jpdemo

sreeaswar

sys

SQL File 3*

SQL File 3* - %

Limit to 1000 rows

```
1 create database event_bookings;
2 use event_bookings;
3 show tables;
4 describe category;
5 select * from category;
6 describe venue;
7 select * from venue;
8 describe availability;
9 select * from availability;
10 describe user;
```

Result Grid

Filter Rows

Edit

Export/Import

Wrap Cell Contents

venue_id	capacity	description	image_url	location	price	services	venue_name
1	500	A spacious air-conditioned venue suitable for w...	https://example.com/images/grand-palace.jpg	Chennai	150000	Catering, Decoration, Parking, DJ	Grand Palace Convent
2	600	A spacious air-conditioned venue suitable for w...	https://example.com/images/chozhamahal.jpg	Chennai	100000	Catering, Parking, DJ	Chozha Mahal
3	1000	A Nice venue suitable for weddings and recepti...	https://example.com/images/ganga.jpg	Chennai	60000	Catering, Parking, Buffet, DJ	Ganga Mahal

venue 2 x

Apply Revert Context Help Snippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	20:25:43	use event_booking	0 row(s) affected	0.000 sec
2	20:25:51	select * from category LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec
3	20:27:06	select * from venue LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec

Object Info Session

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

MySQL Workbench

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHMAS

Filter objects

course_req_sym

Tables

Views

Stored Procedures

Functions

event_booking

Tables

Views

Stored Procedures

Functions

joademo

sreeeaswar

sys

Administration Schemas

Information

No object selected

SQL File 3*

SQL File 3*

Limit to 1000 rows

```
4 * describe category;
5 * select * from category;
6 * describe venue;
7 * select * from venue
8 * describe availability;
9 * select * from availability;
10 * describe user;
11 * select * from user;
12 * describe cart;
13 * select * from cart;
```

Result Grid

user_id	password	role	Username
1	the6784@viedft	VenueOwner	Street Easwar
2	the6784@viedft	VenueOwner	Vignesh
3	thyviedft	Customer	Salman
4	77h67	Customer	Roshan
5	77h67@de4	Admin	Ramesh
6	77h67@de4	Admin	Ramesh

user 4 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	20:25:43	use event_booking	0 row(s) affected	0.000 sec
2	20:25:51	select * from category LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec
3	20:27:06	select * from venue LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
4	20:28:19	select * from availability LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
5	20:28:34	select * from user LIMIT 0, 1000	6 row(s) returned	0.000 sec / 0.000 sec

Apply Revert Context Help Snippets

Object Info Session

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

2028 08-08-2026

MySQL Workbench

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHMAS

Filter objects

course_req_sym

Tables

Views

Stored Procedures

Functions

event_booking

Tables

Views

Stored Procedures

Functions

joademo

sreeeaswar

sys

Administration Schemas

Information

No object selected

SQL File 3*

SQL File 3*

Limit to 1000 rows

```
1 * create database event_booking;
2 * use event_booking;
3 * show tables;
4 * describe category;
5 * select * from category;
6 * describe venue;
7 * select * from venue
8 * describe availability;
9 * select * from availability;
10 * describe user;
```

Result Grid

availability_id	available_slots	date	total_slots	venue_id
1	49	2025-08-15	50	1
2	10	2025-08-15	10	2
3	7	2025-11-12	8	3

availability 3 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	20:25:43	use event_booking	0 row(s) affected	0.000 sec
2	20:25:51	select * from category LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec
3	20:27:06	select * from venue LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
4	20:28:19	select * from availability LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec

Apply Revert Context Help Snippets

Object Info Session

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

2028 08-08-2026

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHMAS

Filter objects

- course_reg_sym
 - Tables
 - Views
 - Stored Procedures
 - Functions
- event_booking
 - Tables
 - Views
 - Stored Procedures
 - Functions
- joademo
- sreeeaswar
- sys

Administration Schemas

Information

No object selected

SQL File 3*

```

7 * select * from venue;
8 describe availability;
9 * select * from availability;
10 * describe user;
11 * select * from user;
12 * describe cart;
13 * select * from cart;
14 * describe booking;
15 * select * from booking;
16

```

Result Grid

cart_id	customer_user_id	booking_date	slot	customer_id	column_id	venue_id
1	1	2026-08-15	Morning	4	2	1

Output

#	Time	Action	Message	Duration / Fetch
1	20:25:43	use event_booking	0 row(s) affected	0.000 sec
2	20:25:51	select * from category LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec
3	20:27:06	select * from venue LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
4	20:28:19	select * from availability LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
5	20:28:34	select * from user LIMIT 0, 1000	6 row(s) returned	0.000 sec / 0.000 sec
6	20:28:52	select * from cart LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
7	20:29:20	select * from cart LIMIT 0, 1000	1 row(s) returned	0.016 sec / 0.000 sec

Object Info Session

2029 08-08-2026

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHMAS

Filter objects

- course_reg_sym
 - Tables
 - Views
 - Stored Procedures
 - Functions
- event_booking
 - Tables
 - Views
 - Stored Procedures
 - Functions
- joademo
- sreeeaswar
- sys

Administration Schemas

Information

No object selected

SQL File 3*

```

10 * describe user;
11 * select * from user;
12 * describe cart;
13 * select * from cart;
14 * describe booking;
15 * select * from booking;
16
17 * USE event_booking;
18
19 * SET SESSION VIEW CACHE = 0;

```

Result Grid

booking_id	booking_date	booking_time	slot	status	total_amount	customer_id	venue_id
1	2026-08-15	2026-08-08 00:51:01.981183	Evening	CONFIRMED	150000	3	1
2	2026-11-12	2026-08-08 00:51:02.001290	Morning	CONFIRMED	60000	3	3

Output

#	Time	Action	Message	Duration / Fetch
1	20:25:43	use event_booking	0 row(s) affected	0.000 sec
2	20:25:51	select * from category LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec
3	20:27:06	select * from venue LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
4	20:28:19	select * from availability LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
5	20:28:34	select * from user LIMIT 0, 1000	6 row(s) returned	0.000 sec / 0.000 sec
6	20:28:52	select * from cart LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
7	20:29:20	select * from cart LIMIT 0, 1000	1 row(s) returned	0.016 sec / 0.000 sec
8	20:29:47	select * from booking LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec

Object Info Session

2029 08-08-2026

6.API Endpoints

FEATURE	METHOD	ENDPOINT	PURPOSE
category	POST	localhost:8080/category	To create a category
category	GET	localhost:8080/category	To get all category
category	GET	localhost:8080/category/{categoryId}	To get particular category
category	PUT	localhost:8080/category	To update a category
category	DELETE	localhost:8080/category/{categoryId}	To delete a category
venue	POST	localhost:8080/venue	To create a venue
venue	GET	localhost:8080/venue	To get all venue
venue	GET	localhost:8080/venue/{venueId}	To get a particular venue
venue	PUT	localhost:8080/venue	To update a venue
venue	DELETE	localhost:8080/venue/{venueId}	To delete a venue
venue	GET	localhost:8080/venue/owner/{ownerId}	To get venue by ownerId
venue	GET	localhost:8080/venue/search/category/{categoryID}	To get venue by category
venue	GET	localhost:8080/venue/search/location/{location}	To get venue by location
venue	GET	localhost:8080/venue/search/capacity/{capacity}	To get venue by capacity
venue	GET	localhost:8080/venue/search/price	To get a venue by price
user	POST	localhost:8080/user	To create a userId
user	GET	localhost:8080/user/{userId}	To get the User
availability	POST	localhost:8080/availability	To Create the availability
availability	GET	localhost:8080/availability	To get the availability
availability	GET	localhost:8080/availability/{availabilityId}	To get availability by id
availability	PUT	localhost:8080/availability	To Update the availability
availability	DELETE	localhost:8080/availability/{availabilityId}	To delete the availability
cart	POST	localhost:8080/cart	To create a cart
cart	GET	localhost:8080/cart	To get all cart
cart	GET	localhost:8080/cart/customer/{customerId}	To get a particular cart
cart	DELETE	localhost:8080/cart/{cartId}	To get a particular cart
booking	POST	localhost:8080/checkout/{userId}	To confirm booking
booking	GET	localhost:8080/customer/{userId}	To get the Booked Venue
booking	GET	localhost:8080/owner/{ownerId}	To get the booked Venue
booking	GET	localhost:8080/admin	To get all booked venues
booking	PUT	localhost:8080/admin/{bookingId}/{status}	To change Booking Status

7.SUMMARY

The Event & Venue Booking Platform backend was developed using Java and Spring Boot to provide RESTful APIs for managing the core operations of the application. The backend includes modules for user management, category management, venue management, availability management, cart operations, checkout, and booking management.

The application uses MySQL as the database and Spring Data JPA/Hibernate for database interaction. The backend follows a layered architecture consisting of Entity, Repository, Service, and Controller layers, making the application organized, maintainable, and scalable.

All the developed REST APIs were tested using Postman with appropriate request bodies, responses, and HTTP methods. Postman was used to verify the functionality and correctness of each endpoint.

Overall, the backend provides the required business logic and APIs for customers to search and book venues, venue owners to manage their venues and availability, and administrators to manage platform data. The backend can be further enhanced in the future by integrating Spring Security, JWT authentication, online payment services, notifications, reviews, and additional search and filtering features.